宁波大学
NINGBO UNIVERSITY

本科毕业设计（论文）

题目：基于可扩展基数树的键值内存数据库设计

IdleKV: Design of a Key-Value In-Memory Database Based on Adaptive Radix Tree

学　　院	信息科学与工程学院
专　　业	计算机科学与技术
班　　级	22 计算机科学与技术 2 班
学　　号	226000581
学生姓名	陈一波
指导教师	辛宇
完成日期	2026 年 05 月 3 日

诚　信　承　诺

我谨在此承诺：本人所写的毕业论文《基于可扩展基数树的键值内存数据库设计》的主体均系本人独立完成，没有抄袭行为，凡涉及其他作者的观点和材料，均作了注释，若有不实，后果由本人承担并愿接受校方的处分。

承诺人（签名）：陈一波

2026 年 5 月 24 日

目 录

摘要 . . . . . **I**

Abstract . . . . . **II**

1 绪论 **1**

1.1 选题背景与意义 1

1.2 相关研究 1

1.2.1 主内存键值数据库 1

1.2.2 主内存事务与并发控制 2

1.2.3 可扩展基数树与有序索引 3

2 系统概述 **4**

2.1 系统设计原则 4

2.1.1 一致性保证 4

2.1.2 Redis 协议兼容性 4

2.1.3 多核并行利用 5

2.2 系统总体架构 5

2.3 Redis Serialization Protocol 实现 6

2.4 核心模块划分 7

3 IO 与线程模型 **7**

3.1 基于 Fiber 的异步 IO 8

3.2 Fiber 调度机制 9

3.3 Shared-Nothing 线程模型 9

4 基于 VLL 的事务框架 **11**

4.1 VLL：超轻量锁机制 11

4.2 IdleKV 事务实现 11

4.2.1 事务注册 12

4.2.2 事务执行 15

4.3 事务执行优化 16

4.3.1 Pipeline 优化 16

4.3.2 单分片命令优化 17

4.3.3 乱序执行优化 17

5 DB 存储结构 **18**

5.1 设计目标与总体组织 18

5.2 通用键值层 18

5.2.1 对象表示与生命周期管理 18

5.2.2 KvStore 的实现 19
5.3 ART 有序索引 20
5.3.1 自适应节点布局与路径压缩 20
5.3.2 有序编码与游标扫描 20
5.3.3 基于子树计数的 Rank 定位 21
5.4 ZSet 对象设计 21
5.4.1 双索引总体方案 21
5.4.2 更新、删除与范围查询 22
5.5 复杂度分析与工程权衡 22
6 系统测试 23
6.1 硬件与软件环境 23
6.2 Redis VS IdleKV 23
6.2.1 Get/Set 24
6.2.2 ZSet 25
6.2.3 结果讨论 26
7 总结与展望 26
7.1 工作总结 26
7.2 不足分析与后续工作 26
8 致谢 28
参考文献 29

题目：基于可扩展基数树的键值内存数据库设计

摘　要

随着多核 CPU 普及和主内存容量持续增长，传统 Redis 单线程执行模型在单机垂直扩展能力以及有序复合对象支持方面逐渐暴露出瓶颈。为此，本文设计并实现了一个面向 Redis 兼容语义的主内存键值数据库原型 IdleKV。系统在运行时层面采用 Shared-Nothing 分片架构，以及基于 Boost.Asio 与 Boost.Fiber 的多线程事件驱动模型，使请求能够尽可能在本地执行上下文中完成；在执行层面引入基于 Very Lightweight Locking（VLL）的事务注册与调度机制，以在多分片访问场景下维持 Strict Serializability；在数据结构层面实现了基于可扩展基数树（Adaptive Radix Tree, ART）的 ZSet 双索引结构，并通过有序编码、游标扫描与 rank 定位支持 ZADD、ZREM 和 ZRANGE 等操作。实验结果表明，在 GET/SET 工作负载下，IdleKV 的总体吞吐达到 Redis 的 1.81～2.57 倍，平均延迟显著降低；在 ZSet 工作负载下，其吞吐最高可达到 Redis 的 3.79 倍，P99 延迟也整体优于 Redis。与此同时，基于 ART 的 ZSet 实现带来了更高的内存占用，说明有序索引性能与空间效率之间仍存在明显权衡。本文工作验证了在 Redis 兼容语义下，将 Shared-Nothing 执行模型、轻量级事务调度机制与 ART 有序索引结合起来构建单机多核高性能键值数据库的可行性，并为后续完善事务能力与容器实现奠定了基础。

关键字　内存数据库；键值存储；可扩展基数树；NoSQL

IdleKV: Design of a Key-Value In-Memory Database Based on Adaptive Radix Tree

Abstract

This thesis presents IdleKV, an in-memory key-value database prototype. It aims to preserve compatibility with Redis semantics while improving single-node multicore scalability and ordered-access performance. As modern servers provide abundant CPU cores and memory capacity, the traditional single-threaded execution model of Redis increasingly limits vertical scalability, especially for workloads that combine high concurrency with ordered data access. To address this issue, IdleKV adopts a Shared-Nothing sharding architecture together with a multithreaded event-driven runtime built on Boost.Asio and Boost.Fiber, so that requests can be processed as locally as possible. On the execution side, the system introduces a transaction registration and scheduling mechanism based on Very Lightweight Locking (VLL) to maintain strict serializability for multi-shard operations. On the storage side, it implements a dual-index ZSet structure based on the Adaptive Radix Tree (ART), and supports ordered operations such as ZADD, ZREM, and ZRANGE through composite-key encoding, cursor-based traversal, and rank-aware positioning.

Experimental results show that, under GET/SET workloads, IdleKV achieves 1.81∼2.57× the throughput of Redis while also reducing average latency. Under ZSet workloads, the throughput improvement reaches up to 3.79×, and the P99 latency is consistently better than that of Redis. At the same time, the ART-based ZSet design incurs noticeably higher memory overhead, indicating a clear trade-off between ordered-index performance and space efficiency. Overall, this work demonstrates the feasibility of combining a Shared-Nothing execution model, lightweight transaction scheduling, and an ART-based ordered index to build a high-performance Redis-compatible key-value store on a multicore single machine.

KEYWORDS In-Memory Database, Key-Value Stores, Adaptive Radix Tree, NoSQL

1 绪论

1.1 选题背景与意义

随着主内存容量增长和成本下降，越来越多的数据管理系统开始将关键工作集乃至全部事务数据驻留在内存中，以减少磁盘访问带来的高延迟开销；在 OLAP 场景中，生产系统也常通过引入内存缓存或内存加速层来降低交互式查询延迟、提升查询性能。其中 Key-Value 内存数据库在高并发、低延迟的数据处理场景中得到广泛应用，如：缓存系统、实时推荐、流式计算以及 AI 基础设施等等[1]。

然而 Redis 作为目前最广泛使用的键值型内存数据库，仍采用单线程的执行模型，这一设计源于早期内存键值存储系统被认为是 I/O 密集型服务，单线程已能够满足性能需求。但在当前多核 CPU 成为服务器标配、网络带宽逐步增加的背景下，Redis 服务想要利用一台服务器上的多核 CPU 处理指令，通常需要依赖集群分片，这无疑从而增加了部署与运维的复杂度。

另一方面，键值数据库的应用场景并不局限于简单的点查和更新。在排行榜、区间检索、前缀匹配、流式状态管理和推荐系统等场景中，系统还需要同时支持按 key 精确定位和按顺序范围访问。传统的跳表或通用树结构虽然能够提供有序能力，但在主内存场景下往往存在指针跳转较多、缓存局部性较差的问题。可扩展基数树（Adaptive Radix Tree, ART）通过路径压缩和自适应节点布局，在保持有序遍历能力的同时提高了内存访问效率，因此成为主内存数据库中一种具有潜力的有序索引结构。

在上述背景下，设计一种兼顾 Redis 协议兼容性、多核扩展能力与高效有序索引结构的主内存键值数据库具有重要的研究价值与工程意义。本文实现的 IdleKV 在我们的基准测试中，使用多核 CPU 在不同工作负载下，吞吐以及平均延迟指标实现相对 Redis 大约 2 倍的性能提升，具体提升参考图 1.1 所示。

1.2 相关研究

主内存数据库、轻量级并发控制和高效有序索引是本文的三个核心技术背景，本小节将详细介绍这些主题当前的研究现状。

1.2.1 主内存键值数据库

随着主内存容量持续增长，越来越多的键值存储系统开始以 DRAM 作为主要数据承载介质，重新审视传统数据库中由磁盘主导的系统设计。Ousterhout 等人提出的 RAMCloud 是较早的代表性工作之一，它主张将数据常驻内存、将磁盘主要作为备份介质，从而在集

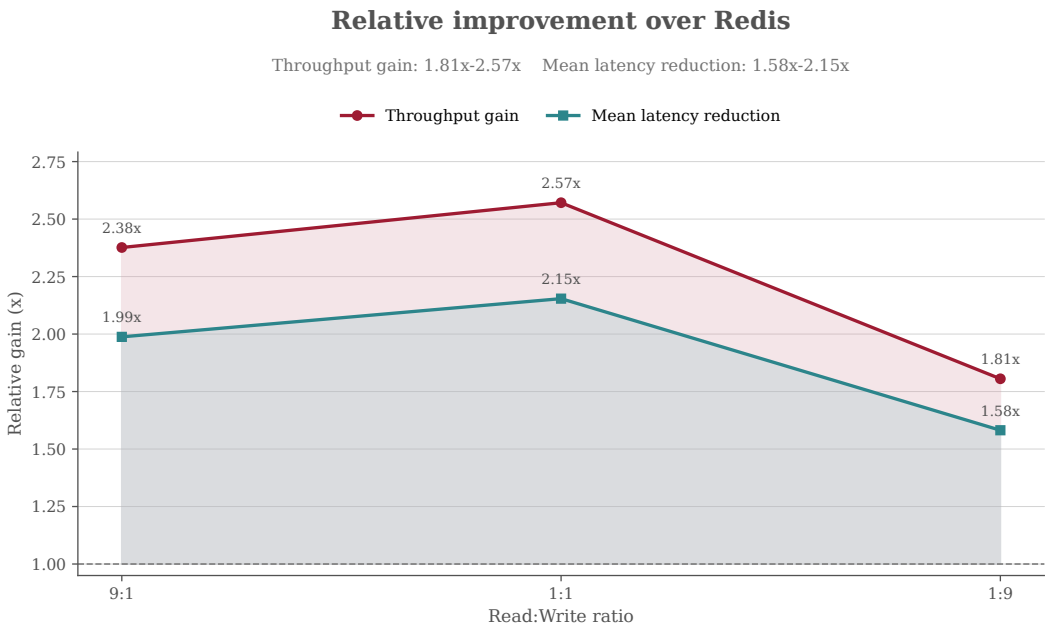

图 1.1: IdleKV 相对 Redis 的基础读写性能提升概览

群环境中提供低延迟、高吞吐的存储服务[2]。这一方向表明，当磁盘 I/O 不再位于关键路径上时，数据布局、网络路径和并发执行模型会成为影响系统性能的核心因素。

在单机高性能键值系统方向，相关研究进一步把优化重点转向了多核并行执行和缓存友好的内存访问。MemC3 通过紧凑哈希表、近似 LRU 的 CLOCK 淘汰策略以及乐观锁等工程优化，显著提升了 Memcached 的内存利用率与并发性能[3]。MICA 则从请求分发、网络收包、数据分区和并行访问等多个层面协同优化，在通用多核服务器上实现了极高的单机吞吐[4]。进一步地，FASTER 利用缓存友好的并发哈希索引与混合日志相结合的设计，优化了点查、更新与 read-modify-write 路径，体现了现代键值存储对高并发更新场景的关注[5]。

总体来看，主内存键值数据库相关研究主要围绕“如何充分发挥内存介质优势”和“如何在多核平台上压缩每次请求的常数开销”两个方向展开。然而，这类系统大多以无序键值访问、点查与更新负载为主要优化目标，对于兼顾 Redis 兼容接口、多核扩展能力和有序范围访问的统一设计关注相对较少，这也构成了本文工作的切入点。

1.2.2 主内存事务与并发控制

事务作为数据库领域最核心的主题之一，相关研究已经非常成熟。为了在保证基础的 ACID 性质的前提下尽可能提高并发度，提出了非常多的并发控制机制，这些机制大致可以分为两类：1. 悲观并发控制；2. 乐观并发控制。

悲观并发控制协议通常基于锁来实现。Two-Phase Locking (2PL) 作为一种基础的锁并发控制协议，它可以保证可序列化的（Serializable）事务一致性模型，最早由 Jim Gray

在早期研究中奠定了 2PL 基本思想，后来由 Bernstein 和 Goodman 在其关于并发控制机制的综合研究中予以系统化[6]。在此基础上，为了在提高事务并发度的，Gray 等人进一步提出了多粒度锁[7]，引入了层次结构的意向锁，在保证可串行化的前提下，有效降低了粗粒度锁带来的并发受限问题。

乐观并发控制协议通常假设事务的冲突很少发生，在真正发生冲突时会重启事务。David P. Reed 在 1978 提出了一种基于时间戳（T/O）的乐观并发控制协议[8]，在单个事务涉及数据不多且冲突较少时，可以节省 2PL 中控制锁的额外成本，提高事务并发度。H. T. Kung 和 John T. Robinson 等人提出在此基础上提出一种更通用的乐观并发控制（Optimistic Concurrency Control, OCC）模型[9]，将事务执行过程划分为读阶段、验证阶段和写阶段，通过延迟冲突检测来提高系统的并发度与执行效率。

传统并发控制协议起源于磁盘数据库时代，其设计目标主要围绕降低磁盘 I/O 开销与提升事务吞吐能力，因此系统瓶颈通常集中于存储访问延迟，并发控制开销相对次要。在主内存数据库中，数据完全驻留于内存，磁盘 I/O 不再构成主要瓶颈，并发控制机制的开销逐渐成为性能关键限制。针对上述问题，Kun Ren 等人提出一种轻量级的锁控制机制（Very Lightweight Locking，VLL）[10]，通过去除中心化的共享/独占计数器与有序事务队列来推进可执行事务，以降低并发控制本身的元数据成本，使其在单核以及多核环境下都有非常优秀的性能表现。

1.2.3 可扩展基数树与有序索引

在有序索引方向，经典的 B^+-tree 仍然是最重要的基线结构之一。它依靠较高扇出和叶子层顺序链接，能够同时支持点查与区间扫描，因此长期被广泛用于数据库系统中。然而，传统 B^+-tree 的节点组织方式主要围绕页式存储设计，在主内存环境下，其性能瓶颈会逐渐从磁盘 I/O 转向指针跳转、缓存失效与字符串比较。针对这一问题，Rao 和 Ross 提出的 cache-sensitive B^+-tree（CSB^+-tree）[11] 通过压缩子指针布局和增强节点连续性，显式地将缓存局部性纳入设计目标，说明即便仍保留 B^+-tree 的整体框架，主内存场景下也需要重新审视节点布局与访存模式。

另一类常见的有序结构是 SkipList。Pugh 提出的 SkipList[12] 以概率平衡代替严格旋转维护，插入与删除逻辑简洁，工程实现也相对直接，因此在许多内存型系统中被用作有序集合或 memtable 的底层结构。Redis 的 Sorted Set 便是采用“Hash Table + SkipList”的典型代表。不过，SkipList 的查找路径本质上仍由多层前向指针跳转组成，其缓存行为较强依赖对象分配布局；当 key 较长或需要进行深范围扫描时，这类随机访问的代价会变得更加明显。

为了进一步兼顾变长键上的公共前缀利用与范围访问能力，Mao 等人提出了一种复

合索引结构（Masstree）[13]。该结构将 trie 与 B^+-tree 拼接起来处理变长键与长公共前缀，并结合缓存友好的节点布局与细粒度并发控制，在多核主内存环境下兼顾了范围访问能力和较高吞吐。这说明主内存环境下的有序索引设计，不能简单沿用面向磁盘的树结构，而需要显式考虑缓存层次、CPU 访存成本以及 key 编码特征。

在此基础上，Leis 等人提出可扩展基数树（Adaptive Radix Tree, ART）[14]，通过路径压缩和 Node4、Node16、Node48、Node256 的自适应节点布局，在空间利用率和缓存局部性之间取得了更好的平衡。与传统 B^+-tree 相比，ART 直接按字节前缀组织键，能够更自然地处理长公共前缀；与 SkipList 相比，它减少了多层链式指针跳转带来的随机访问开销。ART 的点查性能接近哈希索引，同时仍然保留了按字典序遍历、前缀匹配和范围扫描等有序能力，因此成为主内存数据库中非常有代表性的有序索引结构。

2 系统概述

本章主要概述 IdleKV 的整体设计原则与系统架构。

2.1 系统设计原则

本文旨在实现一套在多核 CPU 上具备良好性能表现的 Redis 替代系统。为兼顾性能与工程可维护性，有必要在设计之初明确系统的整体设计原则与核心理念。

2.1.1 一致性保证

Redis 之所以能够在工程上较容易地提供强一致的可观察行为，一个重要原因在于其核心命令执行路径基本是串行化的。作为面向 Redis 语义的多核替代系统，IdleKV 不能只追求吞吐提升，还需要在并发访问场景下维持与 Redis 接近的事务可见性。因此，本文将 Strict Serializability 作为系统的一致性目标[15]，要求每个成功提交的操作既满足可串行化，又满足与真实时间顺序一致。

本文使用 Elle[16] 作为黑箱事务一致性检查器，对 IdleKV 生成的事务历史进行离线校验，用于验证当前实现是否满足预期的一致性模型。

2.1.2 Redis 协议兼容性

系统对外需要尽可能保持 Redis 风格的访问接口，使现有客户端、基准测试工具和运维脚本能够低成本迁移。为此，IdleKV 选择以 Redis Serialization Protocol（RESP）作为唯一网络交互协议，并尽量保持命令参数组织方式、返回值类型以及错误语义与 Redis 一致，从而降低系统替换与实验对比的门槛。

考虑到论文原型实现的范围，初版系统优先支持 RESP2，并围绕字符串命令、基础系统命令以及 Sorted Set 相关命令构建完整处理链路。这样的取舍既能够覆盖本文关注的核心工作负载，也为后续继续补齐更完整的 Redis 命令集留下了清晰扩展接口。

2.1.3 多核并行利用

提高多核 CPU 利用率是本文系统设计的核心目标之一。随着单机服务器核心数量不断增加，键值数据库的性能瓶颈不应长期集中在单个执行单元上，而应尽可能将并发请求转化为可并行处理的工作负载。对于主内存数据库而言，单次请求的计算路径通常较短，若系统结构无法有效分摊这些请求，即使硬件提供了更多 CPU 核心，整体吞吐也难以随之提升。因此，本文在设计上应按照“局部优先、并行推进、按需协调”的设计思路，减少夸核心数据迁移、缓存失效和同步带来的额外开销。

2.2 系统总体架构

传统多线程系统通常依赖锁机制或原子操作来实现并发控制，然而，随着线程数量的增加，资源竞争显著加剧，导致系统性能难以随着 CPU 核心数的增长实现线性扩展。为此，IdleKV 采用 Shared-Nothing[17] 架构，将系统划分为 N 个相互独立的分片，其中 $N \leq$ CPU 核心数。每个分片由单一线程独占访问，并通过线程与 CPU 核心的绑定（CPU pinning）减少线程迁移带来的缓存失效开销。在该设计中，每个线程同时负责 TCP 连接处理以及对应数据库分片的管理，从而避免跨线程的数据访问与同步开销。

在每个线程之上，IdleKV 采用更轻量级的 Fiber（协程）来执行不同的任务（如：数据操作执行、客户端连接处理以及异步 IO 等）。相比于操作系统线程，Fiber 具有更低的创建与上下文切换开销，其调度完全由用户态控制，从而避免了内核调度带来的额外开销。此外 IdleKV 能够根据系统特性设计调度策略，以此来提升线程内部的执行效率，增强系统在高负载场景下的可控性。详细内容将在第 3章展开。

如图 2.1 所示，IdleKV 在示例配置中创建了 3 个工作线程，每个线程独立负责其分片内的 I/O 处理与数据管理。当新的客户端连接到达时，系统会将对应的客户端协程（fiber）分配至某一线程执行。线程选择由调度策略决定，默认采用 Round-Robin（轮询）方式以实现负载均衡。在 Linux 环境下，若配置了 RPS（Receive Packet Steering），IdleKV 可进一步将连接调度至处理该网络数据包的 CPU 所对应的线程，从而提升 CPU 缓存局部性，优化整体性能。

客户端的命令可能涉及到不同分片的数据，要保证多分片操作执行的全局顺序一致，就需要经过事务协调后，才能调度到对应分片的线程上执行数据操作。这里事务协调基于 VLL[10] 并发控制机制来实现事务的全局顺序一致性，详细内容将在第 4 章中展开。

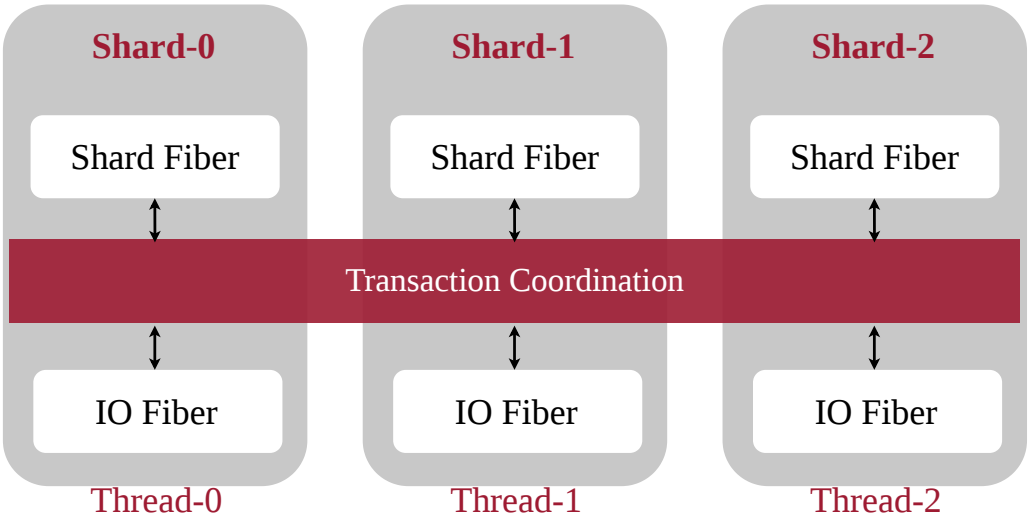

图 2.1: IdleKV 整体架构图

2.3 Redis Serialization Protocol 实现

RESP 是一种支持多种数据类型的二进制序列化协议，其控制序列使用标准 ASCII 编码。在 RESP 中，数据的第一个字节决定了其类型。当前 IdleKV 实现了 RESP2 中覆盖命令解析与结果返回所需的主要数据类型，其支持范围如表 2.1 所示。

表 2.1: IdleKV 当前支持的 RESP2 数据类型

数据类型	前缀/形式	支持位置
Simple String	+	响应
Error	-	响应
Integer	:	响应
Bulk String	$<len>	
Array	*<len>	请求、响应
Null Bulk String	$-1	响应

IdleKV 同 Redis 相同，使用 RESP 作为 request-response 请求-响应协议，其工作方式如下：

1. 客户端以“批量字符串数组”（array of bulk strings）的形式向 Redis 服务器发送命令。其中，第一个字符串表示命令名称，其后的元素则作为该命令的参数。

2. 服务器则以 RESP 格式返回响应，响应的数据类型由具体命令的实现决定。

IdleKV 实现了高效的 Zero Copy 的 RESP 协议 Parser 以及 Sender。

首先是命令解析使用的 Parser，对于协议的数据类型、字符串长度和数组长度这样的头部数据，Parser 使用一个固定大小的缓冲区来处理，避免频繁的小内存分配，对于命令以及参数这样的数据本身，会被直接读取到一个已经提前申请好空间的 CmdArgs 类型

中，在后续的所有操作中不会对 CmdArgs 中的参数做任何的移动以及拷贝。

之后是用于响应恢复的 Sender，我希望尽可能减少 write 系统调用的次数，尽可能批量的发送响应，以此提高整个系统的吞吐，因此 Sender 内部有一个可扩容的写缓冲区，只有当缓冲区写满以及显示调用 Flush 时才调用 write 发送响应数据，这个对吞吐的提升是显而易见的，在实现过程中添加此优化后系统吞吐有 2~3 倍的提升。

然而，对于 GET、MGET 等读操作而言，若将返回的 Value 数据拷贝至缓冲区再统一发送，会引入一次额外且可避免的内存拷贝。为此，Sender 在处理此类请求时仅持有 Value 数据的内存地址，而不将其复制至内部缓冲区。在实际发送阶段，采用 writev 系统调用将多个非连续内存区域（如协议头部与 Value 数据）组织为一组 iovec 结构进行聚合写入，从而在避免额外拷贝的同时，仍然保持批量发送的优势。此外，Sender 在持有 Value 内存引用期间，需要保证其生命周期的有效性。IdleKV 中 Value 采用 std::shared_ptr 进行管理，因此 Sender 仅需持有一份指针拷贝，即可确保其在发送完成前不会被提前释放。

2.4 核心模块划分

IdleKV 中的核心模块还需要分章节展开详细说明，主要是如下三个部分：

基础设施 为 Shared-Nothing 架构提供底层运行时支持，包括线程管理、网络 I/O、多路复用机制以及 Fiber 调度与同步原语等。

VLL 事务框架 系统实现 Strict Serializability 一致性语义的核心模块，通过全局顺序控制与冲突管理机制，保证事务执行结果等价于某一全局串行顺序。

底层 DB 数据结构 负责各类数据结构的具体实现，包括 KV 存储（KvStore）、有序集合（ZSet）等。

3 IO 与线程模型

IdleKV 采用“线程级并行 + 线程内异步”的总体线程模型。系统通过多个工作线程利用多核 CPU 的并行能力，而在每个线程内部，则借助 Fiber 与事件循环实现 Proactor 风格的异步 IO 模式。需要特别说明的是，异步性（asynchronicity）与并行性（parallelism）并不等价：前者强调单个线程在等待 I/O 时不被阻塞，后者强调多个 CPU 核同时推进不同任务。IdleKV 同时利用了二者，即跨线程实现并行，在线程内部实现异步。

在这一模型下，连接处理、分片任务执行等待任务都与一个 Fiber 绑定，Fiber 间采用主动让权的协调式 CPU 调度机制，Fiber 在遇到阻塞任务时或在一些条件下（如：到达 CPU 使用配额上限）就会主动放弃 CPU，由同线程中的其他 ready Fiber 继续运行。

3.1 基于 Fiber 的异步 IO

IdleKV 选择了 Boost.Asio 以及 Boost.Fiber[18-19] 来为系统提供异步 IO 以及协程调度的能力。Boost.Asio 在调用异步操作时需要传入的 completion token 来生成对应的 async handler（在异步任务完成时执行后需要执行的任务）以及 async result（异步操作的返回值操作）。通常 async handler 是一个回调函数，但逻辑复杂时会出现回调函数过多，核心逻辑混乱的问题，也不能和 Boost.Fiber 很优雅的结合在一起。

```
while (true) {
    nread = async_read(socket, buf, yield);
    res = process_input(buf, nread);
    nwrite = async_write(socket, res, yield);
}
```

Code 1: 使用 yield_t 的同步风格 I/O 示例

为此，IdleKV 扩展了一种新的 completion token：yield_t，并实现了 yield_handler 负责将 fiber 挂起与唤醒。同时，还扩展实现了 async_result 来返回异步操作的结果，把 fiber 的挂起与唤醒操作嵌入到异步流程之中，使异步代码能够以接近同步代码的方式书写如 Code 1 所示。

从代码表面上看，上述逻辑与阻塞式 socket 编程非常接近；但实际上，当执行流到达 async_read() 和 async_write() 时，异步操作流程就如图 3.1 所示。

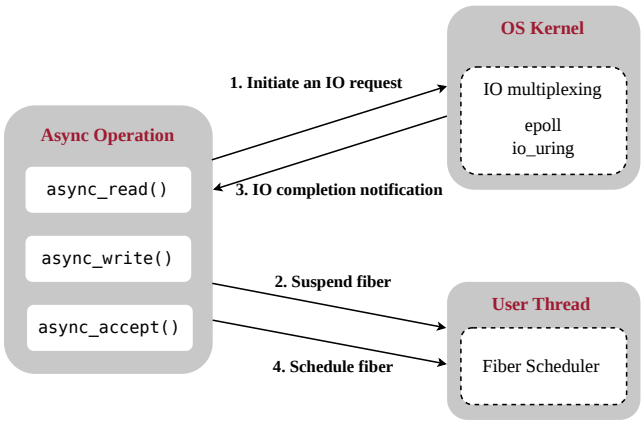

图 3.1: 基于 Fiber 的异步工作流程

1. 注册 IO 请求；2. 挂起协程；3. IO 完成；4. 唤醒 fiber 继续执行。

3.2 Fiber 调度机制

同一个线程上会允许各种各样的类型的任务，如：客户端请求处理、分片任务队列。为了最小化客户端的响应延迟，当一个客户端对一个分片发起读写请求时，我们希望尽可能快的执行对应操作。为此系统在事件循环线程中启用自定义优先级调度器，并为 Fiber 定义 HIGH、NORMAL 和 BACKGROUND 三个优先级，使 shard 任务队列消费者、连接处理和后台任务能够在同一线程内获得不同调度权重。

仅有优先级还不足以保证线程内公平性。协调式调度的一个风险在于：如果某个 Fiber 长时间不主动让出 CPU，那么即使线程内部仍有其他 ready Fiber，也无法及时获得执行机会。为此，IdleKV 利用硬件 cycle counter 为热路径设置 CPU 预算，并在超过预算后触发主动 yield。其中，解析阶段和 Pipeline 聚合阶段分别由 kMaxParseCpuCycles 与 kMaxSquashCpuCycles 两个阈值约束，从而缓解单个长任务对线程内尾延迟的拖累。

3.3 Shared-Nothing 线程模型

IdleKV 的扩展性首先来自于 Shared-Nothing 分片模型。系统使用 XXHash64[20] 将 key 按哈希结果划分到不同的 shard 中，每个 shard 长期绑定到一个工作线程，并由该线程独占访问。这样一来，绝大多数单 key 命令从网络解析、路由判定到 DB 访问都可以在线程本地完成，线程间只需要共享极少的调度信息，不需要在热路径上引入全局锁或频繁的原子同步。由于同一个分片内部始终保持单线程执行，KvStore、ART、ZSet 等数据结构也不必再为分片内并发访问额外维护细粒度锁，这不仅减少了同步开销，也显著降低了实现复杂度。

在线程内部，IdleKV 的每个工作线程既运行 Asio 事件循环，也负责所属 shard 的任务消费与事务推进。线程本地会维护自己的任务队列、事务队列、逻辑 DB 切片以及独立的内存池。

IdleKV 采用 mimalloc[21] 作为内存分配器，其线程本地化（thread-local）的设计与系统的 Shared-Nothing 架构高度契合。相比 glibc malloc、jemalloc 以及 tcmalloc 等传统分配器，mimalloc 通过为每个线程维护独立的堆，并结合延迟回收（deferred remote free）机制，有效降低了跨线程内存操作带来的竞争开销。此外，该设计还能够提升缓存局部性（cache locality），从而减少跨核访问带来的性能损耗，并在高并发场景下显著改善系统的尾延迟表现。

但对于 MGET、MSET、DEL 等可能会跨 Shard 的多 Key 命令，IdleKV 仍需要一些跨线程的调度协作。参考图 3.2 所示的例子，请求首先由连接所在的线程接收并解析。图中 Connection 位于 Shard-n 对应的线程中，因此该线程既负责维护客户端连接，也负责

发起这次跨 Shard 命令的调度。完成 key 路由后，系统会将属于 Shard-n 本地的数据访问组织为本地任务，同时把涉及 Shard-m 的子任务封装为调度单元（在 IdleKV 的实现中就是一个可调用对象），投递到目标线程的 TaskQueue 中。这样一来，跨 Shard 命令虽然由一个连接统一发起，但真正的数据操作仍然会被拆分后下沉到各个目标 shard 的本地执行上下文中。

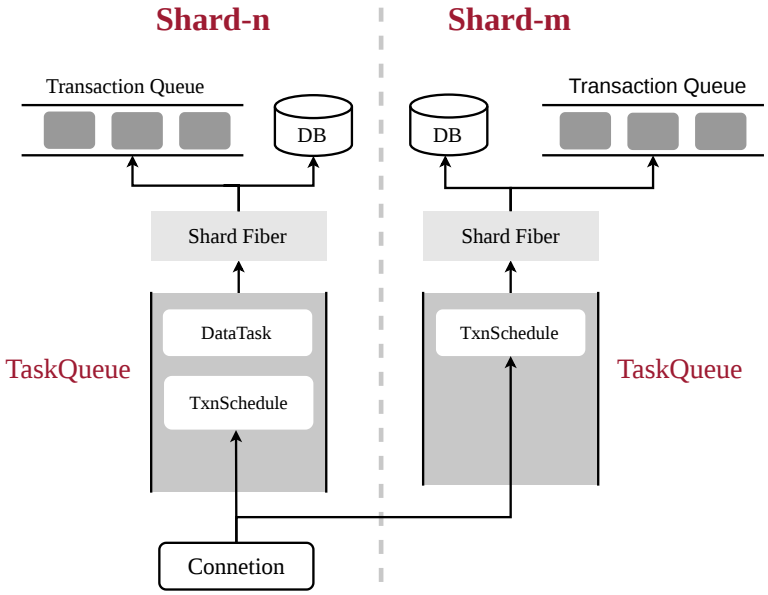

图 3.2: 跨 Shard 调度图

通常 TaskQueue 中的任务有两类：1. 数据读写操作；2. 事务调度操作。对于单分片命令可以直接发送其对应的数据操作到目标分片，但涉及多个分片的命令，在执行数据操作之前需要成功的在所有涉及到的分片中的事务队列中调度成功，详细逻辑将在第 4章展开。

从客户端视角看，这类跨分片命令依然表现为一次完整的请求-响应过程。连接线程负责聚合各个目标分片的执行结果，并在所有相关子任务完成后统一回复响应。

由上述过程可以看出，在 Shared-Nothing 执行路径中，TaskQueue 是承担跨线程任务传递的核心结构。当前 IdleKV 使用了 Boost.Fiber 提供的 buffered_channel，将其作为 Fiber 友好的多生产者单消费者队列（MPSC Queue）来实现 TaskQueue。但其仍需要锁来同步多线程操作，后续可以考虑以更轻量的无锁队列替换现有实现。

4 基于 VLL 的事务框架

对于 Redis 这类采用单线程执行模型的系统，由于所有命令在同一执行序列中按顺序处理，因此天然具备原子性，并能够满足 Strict Serializability 的一致性要求。而在多线程执行模型下，不同线程上的操作可能并发发生，如果缺乏统一的调度与协调机制，则难以保证全局执行顺序的一致性，因此需要引入额外的事务控制框架。

IdleKV 基于 VLL[10] 实现了 Shared-Nothing 架构的事务机制。该机制通过为事务分配全局有序的 $\mathrm{txn_id}$，并在各个分片上按照该顺序推进执行，从而确保跨分片事务在逻辑上等价于按同一全局顺序串行执行。

4.1 VLL：超轻量锁机制

在 VLL 中，传统的锁结构被简化为两个计数器 (C_X, C_S)，分别表示当前记录上持有排他访问和共享访问的事务数量。事务在分片中注册时，需要请求（Acquire）对应类型的锁，就是对相应 key 的 C_X 或 C_S 加 1，具体取决于锁的类型（Exclusive 或 Shared）。如果请求之后满足 $C_X = 1$ 且 $C_S = 0$，那么这个事务就被认为已经获得了排他锁；如果请求之后 $C_X = 0$，该事务就被认为已经获得了共享锁。如果一个事务成功的获取了所有键的锁，那么该事务会被标记为 $Free$，相反的该事务会被标记为 $Blocked$。

当事务请求了所有 key 的锁后，无论事务是否是 $Free$ 的，都会被加入到分片中的 $\mathrm{TxnQueue}$ 中。由此很容易得出如下结论：1. 在队首的事务一定是可以执行的；2. 被标记为 $Free$ 的事务无论在不在队首都可以执行。后者对于实现乱序执行优化很有帮助。

在 IdleKV 中为了进一步减少 $\mathrm{LockTable}$ 的开销，事务会将每个 key 转化为一个 64 位的数据指纹作为获取锁的键，来减少字符串的内存占用以及比较开销。

4.2 IdleKV 事务实现

在 IdleKV 中，事务的执行可以概括为两个步骤：1. 注册事务到所有涉及的分片中；2. 执行对应数据操作。一个事务的流程就可以被描述为图 4.1 的样子。

IdleKV 的 Shared Nothing 架构不允许事务直接访问分片数据以及对应的 $\mathrm{TxnQueue}$。相反，事务需要向分片发送请求，并指示它们在每个步骤中该做什么。每次事务发送消息时，都会阻塞直到收到回复。为了后文描述方便，我将这一次请求回复的过程称作一次**调度轮次（Schedule Round）**。

注意，只有事务所在 Fiber 被阻塞时，它所在的线程仍然可以执行其他 Fiber（比如处理其他连接上的请求）。这就是采用 Fiber 的优势，它让 IdleKV 能够将执行上下文与操作系统线程分离。

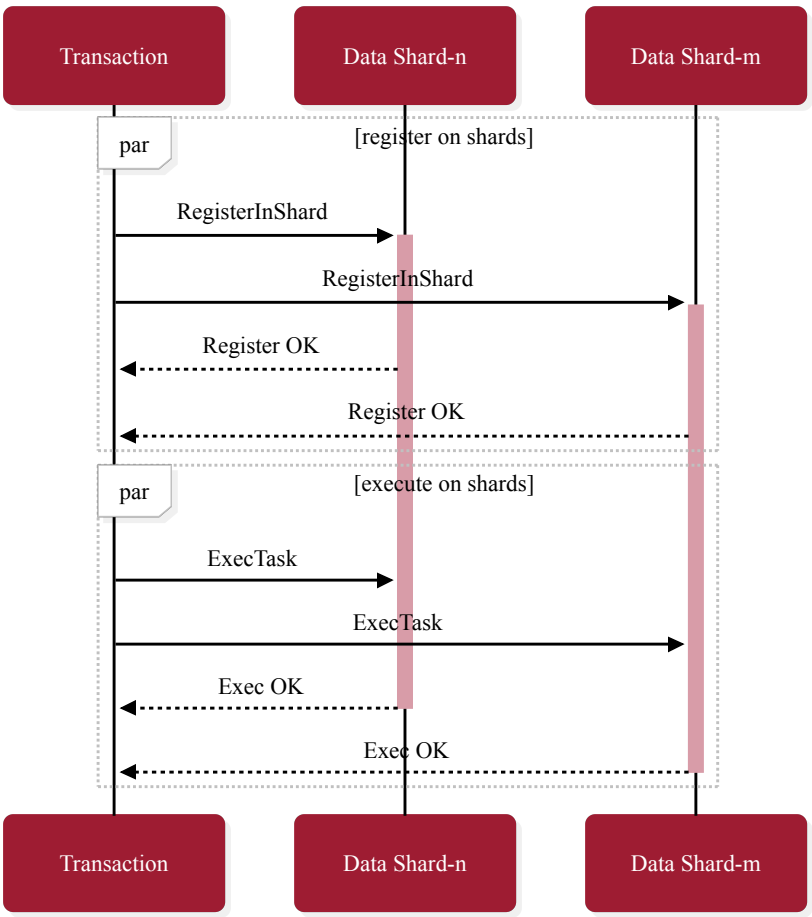

图 4.1: 事务调度执行流程

4.2.1 事务注册

在事务真正开始执行前，需要经过一次 Schedule Round 的注册操作，其目的是为所有相关 shard 建立一致的局部顺序与锁状态。当数据分片接收到注册请求后，会将该事务加入本地的事务队列（TxnQueue）中。为了保证可串行化（serializability），即确保所有分片以一致的顺序对事务进行排序，IdleKV 使用一个全局的原子变量，作为事务的全局序列计数器，用于为所有事务分配一个全序关系。在每个分片中，事务队列（TxnQueue）中的事务按照其序列号进行排序。

但在一些情况下会出现两个分片线程接收到事务的顺序可能并不一致。如图 4.2 示例所示，T1 和 T2 的键值相同，且涉及到 Shard1 以及 Shard2 两个分片中的数据。当 T1 和 T2 同时开始调度时，就会出现了序列号较小的事务在序列号较大的事务之后被插入到队列中的情况。对于 Shard1 我们没办法把 T1 直接插入到 T2 之前，因为 T1 没有成功获取到全部 key 的锁，T1 没办法确定 T2 是否在已经进入到了执行阶段，如果 T2 已经进入执行阶段，就会导致 T1、T2 在不同 Shard 上执行顺序不同。

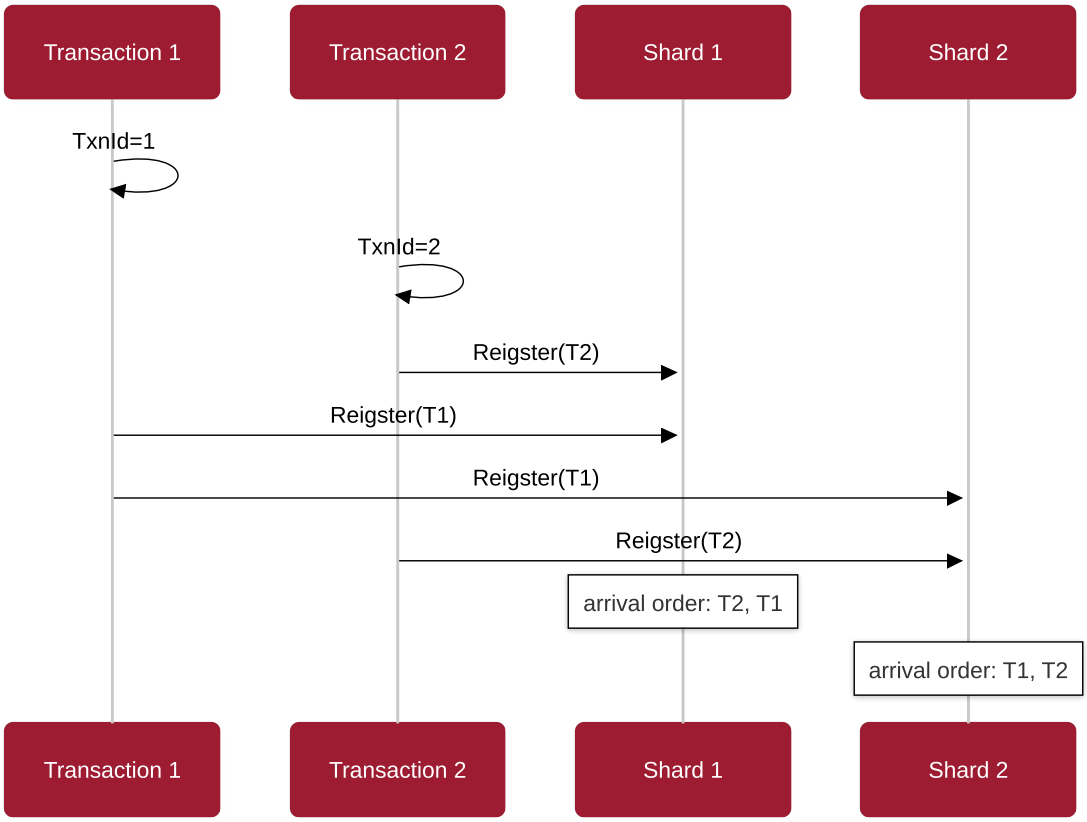

图 4.2: 多分片事务顺序不一致情况流程图

在这种情况下，协调者需要回滚此次注册操作，即从各个分片中移除该事务，并重新分配新的序列号以重试整个注册过程。在实际运行中，注册失败的概率较低，重试也较为少见（主要取决于键上的竞争程度）。需要注意的是，当多个事务同时调度跨分片事务时，才更容易出现这种顺序不一致的问题。

如 4.1 小节中介绍的，事务会在被加入到 TxnQueue 前，请求到该事务涉及的 key 的锁，这些锁不会阻塞底层操作的执行（它们只是两个计数器），而仅用于表达该事务将要访问或修改相应键的意图。当所有注册阶段完成后，VLL 机制就可以在保证 Strict Serializability 的前提下，确保后续执行过程能够持续正确推进。

下面说明 IdleKV 注册流程的具体实现，以及一些工程上的优化。首先需要明确的是事务需要执行的任务会以回调函数 cb 的形式记录在事务内部，当满足执行条件后，就会在对应分片上执行该回调函数，对应事务在该分片上的任务也就执行完成了。

代码 2 展示了将事务注册进一个分片的详细逻辑。核心逻辑就是获取该分片上的锁，再将事务插入本地 TxnQueue，等待后续统一推进执行。但是这里有很多可以优化的细节：

1. 首先注册前要检查当事务号是否已经落后于该分片的提交边界，判断这是否是一次过时的注册，这样的事务时一定不会注册成功的，所以可以避免一些无意义的加锁操作。

```
bool RegisterInShard(shard) {
    if (txn_id != InvalidTxnId && shard.CommitedId() > txn_id)
        return false;

    plan = shard_plans[shard.Id()];
    txq  = shard.TxQueue();
    db   = shard.DbAt(db_index);

    key_locked = db.AcquireTxnLocks(plan.read_fps, plan.write_fps);
    plan.flag |= LockAcquired | (key_locked ? Free : Blocked);

    if (key_locked && active_shard_count == 1) {
        cb(this, shard);
        plan.flag |= OptimizeExecution;
        db.ReleaseTxnLocks(plan.read_fps, plan.write_fps);
        plan.flag &= ~LockAcquired;
        return true;
    }

    if (txn_id == InvalidTxnId)
        txn_id = engine.NextTxnId();

    if (!txq.Empty() &&
        txq.Back()->TxId() > txn_id &&
        !key_locked) {
        db.ReleaseTxnLocks(plan.read_fps, plan.write_fps);
        plan.flag &= ~LockAcquired;
        return false;
    }

    plan.qp_pos = txq.Insert(this);
    return true;
}

```

Code 2: RegisterInShard() 的本地注册逻辑

2. 若当前分片上所需的锁已经一次性获取完成，且事务只涉及单个分片，说明这个事务不需要和其他分片同步顺序，在当前分片也已经达成了可执行的条件，因此系统可以直接执行事务回调来完成任务，可以避免额外一次 Schedule Round。走了这个快速路径的事务会被标记为 OptimizeExecution，事务进入执行期前会用到，下一小节详细说明。

3. 为了保证单分片事务可以稳定调度成功，IdleKV 将单分片事务 txn_id 的获取推迟到该注册操作正在被分片协程执行的时候，这样可以确保该事务的 txn_id 在此时的分片中是最大的，也就能保证其一定注册成功。

4.2.2 事务执行

当事务已经在所有相关 shard 上完成注册后，系统会进入真正的执行阶段。此阶段的事务不会出现任何回滚以及重试操作。每个 Shard 上的事务通过 PollTransaction() 来推进事务回调函数的执行。当事务在所有相关分片上注册完成后，会在每个相关分片上调用一次 PollTransaction()，推进对应分片的事务队列，以此来执行对应事务的回调函数，事务会阻塞等待所有分片上的回调都成功执行。

PollTransaction() 是本地推进事务的核心函数，其核心逻辑参考代码 3。，它首先从 TxnQueue 队首开始扫描，获取已就绪的事务，更新本分片的 commit_txn_id_，并调用该事务的回调函数。ExecCbInShard() 会在当前分片上执行对应的回调函数，并在执行完成后把该事务从 TxnQueue 中移除，释放在当前分片中持有的读写锁，并通过一个 BlockCounter 的同步原语通知调用方该 shard 的执行已经结束。对于 MGET、MSET、DEL 这类跨分片命令，每个分片只负责处理属于该分片的 key 子集，而原始事务所在的连接线程则在所有目标分片都完成后再统一汇总结果并返回响应。

```
void PollTransaction(caller) {
    can_exec_by_myself =
        caller &&
        caller.IsFreeIn(id) &&
        caller.GetExecutionPermit(id);

    while (!tx_queue.Empty()) {
        cur = tx_queue.Front();
        should_run =
            (cur == caller && can_exec_by_myself) ||
            cur.GetExecutionPermit(id);
        if (!should_run)
            break;

        if (cur == caller)
            can_exec_by_myself = false;

        assert(cur.TxId() > commit_txn_id);
        commit_txn_id = cur.TxId();
        cur.ExecCbInShard(this);
    }

    if (can_exec_by_myself)
        caller.ExecCbInShard(this);
}

```

Code 3: PollTransaction() 的本地推进逻辑

要注意事务的回调函数不一定会被自己所调用的 PollTransaction() 执行。所以为了

保证每个回调函数只会被执行一次，每个事务会在完全就绪后依次向每个相关分片上下发一个一次性执行权限（在实现中它是一个原子变量 execution_permit），第一个获取到该权限的 PollTransaction() 调用者才可以执行对应事务的回调函数。对于在事务注册阶段被标记为 OptimizeExecution 的分片是不需要再被执行的，所以在下发 execution_permit 时会跳过对应分片。

如果不是 $Free$ 状态的事务，就只能通过推进事务队列，来尝试执行自己的回调函数，在这个过程中如果遇到了还未就绪的事务，就只能等待其他事务来继续推进。如果是 $Free$ 状态的事务，就会在一开始尝试获取执行自己回调函数的权限，不管事务队列中是否还有未就绪的事务，都可以在本次 PollTransaction() 中完成回调函数的调用。因为队列前面没有和 caller 有依赖关系的事务，执行顺序就可以不按照 txn_id 的顺序，也可以保证对外状态的正确性。

这种“局部执行、集中等待”的方式并不会把整个线程阻塞住。事务 Fiber 在等待注册完成或等待各 shard 执行结束时，只是当前 Fiber 被挂起；对应工作线程上的事件循环、连接处理 Fiber 以及其他 shard 任务仍然可以继续运行。也正因如此，IdleKV 能够在保持同步风格事务代码结构的同时，把真正的等待代价限制在 Fiber 级别，而不是操作系统线程级别。

4.3 事务执行优化

4.3.1 Pipeline 优化

对于 Shared Nothing 这样的架构，在响应 Pipeline 模式下的客户端时会有一个严重的性能问题，那就是每一条命令都需要走一遍昂贵的 Schedule Round。在观察了命令流之后，可以发现对于一系列只涉及到一个分片的命令，它们有两条性质：1）每个命令只需要相对于访问同一分片的其他命令保持其顺序即可；2）各个分片的命令序列可以并行执行。

由此 IdleKV 实现了一套命令聚合逻辑，其基本思想是识别连续的一系列单分片命令，并按分片将它们分开，同时保持它们在每个分片中的相对顺序。一旦命令被分开，我们可以在所有相关分片上执行一次 Schedule Round。在每个分片内，跳转回调将依次执行仅分配给其各自分片的命令。由于所有命令已经放置在其相关线程上，因此不需要任何额外的 Schedule Round，所有命令回调都会内联执行。

在 IdleKV 当前代码中的对应实现是 CmdSquasher。在 DispatchManyCmd() 中，系统会先创建一个父事务，并把其工作模式设置为 MultiMode::Squash。随后，系统通过 CmdSquasher 检查每条命令涉及的所有 key 是否都落在同一个 shard 上；若是，则把该命令追加到该 shard 对应的 MultiSub 子事务中，并用 order_ 记录原始响应顺序；若命令无

法安全聚合，则先冲刷当前批次，再回退到普通事务路径逐条执行。

真正的批量执行发生在 ExecuteSquash() 中。该函数不会让每条命令都单独经历一次完整的“注册全部 shard、设置执行许可、等待完成”流程，而是对每个活跃 shard 只投递一次任务，由目标线程上的 worker fiber 顺序执行该 shard 子事务中的全部命令。由于子事务处于 MultiMode::Squash 模式，命令内部调用 Transaction::Execute() 时会直接在本地 shard 上运行回调，从而跳过常规事务注册过程。所有响应先由 ReplyCapturer 捕获为 Payload，等所有 shard 的批次执行完毕后，再按照 order_ 中记录的原始顺序统一回放给客户端。这样既压缩了跨线程 hop 的数量，也避免了 pipeline 内部重复的事务初始化与队列调度开销。

4.3.2 单分片命令优化

对绝大多数只访问一个 key 的命令而言，完整的跨 shard 事务流程其实过于昂贵。为此，IdleKV 在 RegisterInShard() 中实现了单分片快速路径：如果事务只涉及一个 shard，且在注册阶段已经一次性获得了本地所需的全部锁，那么系统会立即执行命令回调，并把该分片计划标记为 OptimizeExecution。此时事务既不会进入 TxnQueue，也不需要等待后续的 execution_permit 广播和 PollTransaction() 调度，执行完成后便直接释放锁并返回。

这一优化的价值在于，它把“注册”和“执行”合并成了同一次 shard 本地调用。对于 GET、SET、单 key ZADD/ZREM/ZRANGE 等高频命令来说，省去一次入队、一次出队和一次额外的调度投递，能够显著降低事务框架本身带来的常数开销。代码中对单分片事务延迟分配 txn_id 的处理，也服务于这一目标：只有当事务无法走快速路径、确实需要进入本地事务队列时，才为其补上事务号并参与排序。

4.3.3 乱序执行优化

VLL 的一个重要观察是：只要某个事务已经成功获得了本地所需的全部锁，那么它在局部上就是“自由的”（Free），并不一定非要等待自己成为 TxnQueue 队首才能执行。IdleKV 当前的 PollTransaction() 保留了这一思路。函数在优先推进队首且已经获得 execution_permit 的事务之后，还会额外检查当前调用者事务是否满足“本 shard 上标记为 Free，且已经获得执行许可”这一条件；若满足，即便它暂时不在队首，也可以直接调用 ExecCbInShard() 在本地乱序执行。

这种优化的收益主要体现在减少队首阻塞带来的空转等待。若某个排在前面的事务由于尚未获得全部执行许可或仍受其他 shard 状态影响而暂时无法推进，那么其后方那些已经在本 shard 获取完锁的 Free 事务，不必继续被动排队，而可以先行完成本地工作。由

于这些事务在本地锁层面已经没有冲突，乱序执行不会破坏 shard 内的隔离要求；与此同时，commit_txn_id_ 和 TxnQueue 仍然为真正需要顺序约束的事务保留了正确的推进边界。因此，这一优化并不是取消顺序，而是在不违反局部可执行条件的前提下，把原本会被阻塞的执行机会提前释放出来。

5 DB 存储结构

如果说前两章关注的是“请求如何被送到正确的执行上下文”，那么本章讨论的则是“数据到达目标 shard 后应当以何种方式驻留在内存中”。对于一个 Redis 兼容的主内存系统而言，底层存储结构至少需要同时满足三类要求：其一，顶层键空间上的点查与更新必须保持较低常数开销；其二，复合对象需要针对自身访问模式建立专门索引，而不能退化为单一通用容器。

5.1 设计目标与总体组织

IdleKV 的每个 shard 都独占一组 DB 实例，每个 DB 负责维护某个逻辑数据库下的键空间。由于同一 shard 上的数据只会被该 shard 所属线程访问，底层容器不必再内嵌额外的互斥锁或原子同步逻辑，其设计重点主要是“缓存友好”“内存局部性”和“可预测的单次操作开销”。

在当前实现中，分片内部的对象、索引节点以及顶层键值项都尽量通过 shard 本地的 pmr::memory_resource 分配。该资源以独立 mimalloc heap 为后端，一方面便于把同一 shard 的数据尽量聚集到本地内存区域，减轻跨核访问与缓存污染；另一方面也使得每个 shard 的内存占用和峰值使用量能够被单独统计。换言之，存储结构本身并不是孤立设计的，而是与前文的分片执行模型和本地调度语义配套出现：数据放在本地，任务在本地推进，分配也尽量留在本地。

5.2 通用键值层

5.2.1 对象表示与生命周期管理

顶层键空间中的 value 统一表示为 PrimeValue。它本质上是一个指向 Value 对象的 shared_ptr。这样做的出发点并不只是“统一类型接口”，更重要的原因在于：在 IdleKV 的发送路径中，响应缓冲区可以直接持有底层 value 的引用，并通过 writev 进行聚合发送；因此，存储层对象的生命周期不能简单绑定到某一次命令调用栈，而需要允许“存储层已完成查找、网络层仍在发送”的短暂重叠。引用计数虽然会带来少量额外元数据与原

子语义成本，但它显著简化了零拷贝响应与对象生命周期之间的协调关系。

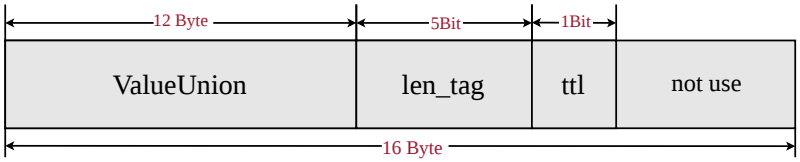

图 5.1: Value 结构示意图

Value 本身采用紧凑的 tagged union 布局，并针对短字符串做了内联优化。当前实现将长度不超过 12 字节的字符串直接保存在对象内部，而较长字符串和 ZSet 这类复合对象则退化为外部分配。这样设计的原因在于，Redis 工作负载中短 key、短 value 占比通常很高，若为每个小字符串都单独分配堆对象，不仅会增大分配器压力，也会显著增加对象指针追踪带来的缓存未命中。通过将小字符串内联到统一 value 容器中，顶层 GET/SET 路径可以在更少的内存访问中完成主要工作；而对于体积较大或结构较复杂的对象，再转交给专门的数据结构实现即可。

5.2.2 KvStore 的实现

顶层键空间当前由 KvStore 承载，其核心容器是 absl::flat_hash_map。选择开放寻址哈希表而非链式哈希表，主要是出于缓存局部性的考虑：开放寻址结构的桶数组更加紧凑，点查时通常只需访问较少的连续内存区域，这对于主内存数据库中的高频短路径操作更为有利。由于 shard 内部已经是串行访问环境，KvStore 无需考虑多线程并发插入和删除，因此可以把注意力集中在查找效率与重哈希成本控制上。

一个更关键的实现细节在于，KvStore 并未采用“一次性完整 rehash”的传统做法，而是将重哈希拆分为渐进式迁移过程。具体而言，容器在扩容后会同时维护旧表和新表，并在每次 Set/Get/Del 时顺带迁移少量元素。这样做的设计动机十分直接：在 Fiber 模型下，单次长时间 rehash 会占用当前 shard 线程较长的 CPU 时间片，进而阻塞同线程上的其他连接与任务推进，最终表现为尾延迟抖动。渐进式 rehash 的代价是实现上要处理“双表并存”以及短期额外空间占用，但它换来了更加平滑的单次操作成本，这一取舍与本文整体强调的“吞吐之外也要避免明显暂停”是一致的。

从逻辑边界上看，DB 并不只是一个纯粹的哈希表包装器。它在顶层键值表之外还维护了事务注册阶段使用的 LockTable，并负责在 Get 路径上执行对象类型检查。因此，DB 承担的是“逻辑数据库入口”角色：哈希表负责对象定位，value 容器负责对象表示，而 DB 负责把类型语义和事务元数据附着在这一层之上。

5.3 ART 有序索引

对于字符串对象而言，顶层哈希表已经足够；但对于 Sorted Set，这样的结构远不能满足需求。ZSet 既要求按 member 做精确更新与删除，又要求按 score 顺序执行范围扫描，特别是在 ZRANGE start stop 中还需要支持从较深的 rank 位置直接开始遍历。若仅使用哈希表，就无法提供有序访问；若仅使用一棵按 score 排序的树，又会使按 member 更新的代价变得过高。

5.3.1 自适应节点布局与路径压缩

当前实现的 ART 采用典型的自适应节点布局，提供 Node4、Node16、Node48 和 Node256 四类内部节点，以及保存最终值的叶子节点。其设计思想是：当某一层分支非常稀疏时，应使用更紧凑的小节点表示，避免像固定 256 路 trie 那样为大量空指针付出空间成本；而当某一层逐渐变得稠密时，再将其提升到更高容量的节点形态，以避免小节点上线性查找的代价持续放大。尤其是 Node48 这一层的存在，使结构能够在“完全稀疏”和“完全稠密”之间提供一个折中点：它用 256 项索引数组记录边值到槽位的映射，同时只保留 48 个真实 child 指针，从而避免直接进入 Node256 所带来的整页指针数组开销。

除了节点自适应布局之外，路径压缩是 ART 在主内存场景中更重要的原因。传统 trie 会为 key 的每一个字节都创建一层节点，面对长字符串或复合 key 时高度很容易膨胀；而当前实现会把“从当前节点继续向下直到分叉发生之前的公共后缀”直接压缩进节点前缀中。对于 ZSet 使用的复合编码 key 而言，这一点尤为关键：相邻元素常常共享相同的 score 高位字节，member 前缀也可能具有较强相似性。若没有路径压缩，树高度和指针跳转次数都会明显上升；引入压缩后，树的实际高度更接近“信息发生分化的位置”，这比简单按原始字节长度建树更适合内存型索引。

5.3.2 有序编码与游标扫描

ART 本身只理解“字节序”而不理解“数值序”或“字符串终止语义”，因此它能否表达 ZSet 所需的有序关系，取决于上层如何编码 key。当前实现通过 ArtKeyCodec 把字符串、整数和浮点数统一转换为“在字节字典序下仍保持原语义顺序”的二进制表示：字符串部分对 0x00 和 0x01 做转义，并在末尾追加终止符，以保证任意字节内容都可安全比较；固定宽度数值采用大端序编码；双精度浮点数则通过符号位变换将 IEEE-754 比特模式映射为可比较的有序字节串。这样一来，ART 可以继续保持“泛化的字节索引”身份，而 score、member 等具体排序语义则被封装在编码规则之中。

基于这一编码层，ART 进一步提供了 Begin()、LowerBound() 和 Cursor::Next() 等有

序扫描接口。这里一个容易被忽略的设计点是：当前实现没有为节点维护父指针，叶子节点也不保存完整 key，因此游标不能像 B+Tree 那样只凭叶子链向前移动。相反，Cursor 必须显式维护一条从根到叶子的路径栈，以及用于重建当前 key 的缓冲区。这样的实现复杂度确实高于“父指针 + 叶子链”方案，但它避免了在更新热路径中维护更多结构性指针，也保留了 ART 节点本身较为紧凑的内存布局。为了简化一致性语义，当前实现还采用 version 检测策略，在树结构发生插入、删除或节点重组后令旧游标失效，从而避免在动态扩缩容后维护复杂的迭代器稳定性规则。

5.3.3 基于子树计数的 Rank 定位

仅有 LowerBound() 对于 ZSet 仍然不够。原因在于 ZRANGE start stop 的核心需求并不是“查找大于等于某个 key 的第一个元素”，而是“查找有序集合中的第 start 个元素”。若只依赖普通游标前进，那么从 start=100000 开始返回 10 个元素时，系统仍然需要执行 10 万次前缀跳过，时间复杂度将退化为 $O(start + k)$。对于深翻页负载，这种线性跳过几乎全部都是无效工作。

为此，当前 ART 实现在内部节点维护了 value_count_ 计数，表示对应子树中叶子节点的数量；叶子节点则隐含记为 1。基于这一计数，SeekByRank() 可以在每一层按 child 顺序累减子树规模，直接下沉到目标 rank 所在的子树，再配合顺序游标继续扫描后续元素。这样，ZRANGE 的访问复杂度就从“先跳过大量前缀元素，再顺序输出”变成了“先按 rank 精确定位，再顺序输出”，近似可写为 $O(h + k)$，其中 h 为树高、k 为返回元素个数。对本文后续的 ZSet 深翻页测试而言，这一设计不是细节优化，而是决定访问路径是否可扩展的关键。

5.4 ZSet 对象设计

5.4.1 双索引总体方案

Sorted Set 的核心矛盾在于：它既是“按 member 精确更新”的字典，又是“按 score 有序扫描”的序列。若只保留按 member 的哈希索引，则范围查询必须额外排序；若只保留按 score 的有序索引，则更新某个 member 的旧分数时又需要先做反向查找。当前实现因此采用双索引结构：score_map_ 负责维护 member 到 score 的映射，score_tree_ 则以编码后的 (score, member) 复合 key 建立 ART 有序索引。前者解决“这个 member 当前分数是多少”的问题，后者解决“按排序语义下一个元素是谁”的问题。

这种设计的优点在于职责划分清晰：哈希索引对点更新友好，有序索引对范围遍历友好，两者都不必勉强承担不擅长的访问模式。其代价也同样明显，即 member 信息会同

时出现在哈希结构和编码索引中，从而放大对象的内存占用。换言之，本文对 ZSet 的设计取向并不是极端压缩空间，而是在可接受的内存代价下换取对更新与扫描两类路径都较稳定的时间复杂度。

5.4.2 更新、删除与范围查询

在插入与更新路径上，ZADD 首先通过 score_map_ 判断 member 是否已经存在。若 member 首次出现，则直接把 member → score 写入哈希索引，并将对应的复合 key 插入 score_tree_；若 member 已存在且分数未变，则无需修改任何结构；若分数发生变化，则先从 ART 中删除旧的 (old_score, member)，再插入新的 (new_score, member)，最后更新哈希表中的分数字段。这个顺序保证了“双索引最终一致”的核心不变量：哈希索引始终能回答 member 的当前分数，ART 始终反映当前有效的排序视图。

删除路径与之对称。ZREM 先通过 score_map_ 找到 member 当前对应的 score，然后从 score_tree_ 中删除复合 key，再从哈希表中移除该 member。当对象元素数降为 0 时，顶层 DB 会继续删除这个 ZSet 对象本身。需要指出的是，这里之所以不需要为双索引之间再引入额外细粒度锁，根本原因不在于代码写得足够小心，而在于 shard 内部的事务执行已经提供了串行化上下文：同一时刻只会有一个本地事务真正修改该对象，因此双索引维护可以在一个连续的本地执行片段中完成。

范围查询路径则体现出 ART 有序索引真正的价值。当前实现首先按照 Redis 语义对 start 与 stop 做标准化处理，包括负下标转换、越界裁剪以及空范围判定；随后通过 CountRange() 预先确定返回元素个数，再调用基于 rank 的 IterateRange() 顺序输出结果。值得注意的是，ZSet 层并不会先构造一个完整的中间结果数组，再交由命令层重新编码为 RESP；相反，它采用回调式输出，把“如何组织协议返回值”的职责留给上层命令实现。这种分层方式的意义在于，容器层只负责有序访问本身，而不直接绑定网络协议格式，从而减少不必要的中间数据物化，也为后续继续优化返回路径留下空间。

5.5 复杂度分析与工程权衡

综合来看，当前存储层在顶层键空间上保持了较为典型的哈希型复杂度：字符串对象的 GET/SET/DEL 在期望意义上均可视为 $O(1)$；ZSet 的 ZADD/ZREM 由“哈希查找 + ART 更新”组成，其代价主要受 member 编码长度和 ART 高度影响；ZRANGE 在引入 rank 定位后，可近似写为 $O(h+k)$，其中 h 为 ART 高度，k 为返回元素个数。就实验所关心的访问模式而言，真正重要的不是将所有操作都压到理论上的最优大 O 形式，而是避免两类在工程上极具破坏性的路径：其一是顶层哈希表扩容时的长时间停顿，其二是深翻页范围查询中的线性跳过。

当然，这一设计并非没有代价。首先，shared_ptr、双索引以及 ART 节点元数据都会明显提高对象的空间开销，尤其在大规模 ZSet 上，这一问题会更加突出：member 既保留在哈希索引中，又参与了 ART 复合 key 的编码，而编码后的 key 进一步被拆分到 ART 的路径前缀、边标签和叶子后缀之中。尽管路径压缩可以回收一部分公共前缀带来的冗余，但由于树节点元数据、前缀存储以及双索引共存，这类开销很难像简单字符串对象那样继续压缩到较低水平。其次，ART 在范围扫描和深翻页上具有明显优势，但其实现复杂度也显著高于跳表、简单有序数组等替代结构，尤其是在节点扩缩容、游标失效语义和子树计数维护方面，需要付出更多工程复杂度。最后，分片本地内存资源改善了局部性和统计能力，却也意味着对象布局更偏向“局部更新、原地演化”，而非依赖后台压缩整理来换取极致空间效率。对于本文的原型系统而言，这种取舍是可以接受的，因为本文更关注的是验证一条面向高吞吐和有序访问的实现路径，而不是在所有对象类型上同时追求最小内存足迹。

6 系统测试

6.1 硬件与软件环境

本文实验均在作者当前开发主机上完成，其硬件与基础软件环境如表 6.1 所示。

表 6.1: 实验硬件与软件环境

项目	配置
处理器	AMD Ryzen 5 9600X，6 核 6 线程，最高主频 5.49 GHz，L3 Cache 32 MiB
内存	DDR5 6000MHz c32 32GB
操作系统	Ubuntu 24.04.3 LTS，Linux 6.14.0-37-generic，x86_64
编译环境	GCC/G++ 13.3.0，CMake 3.28.3，Ninja 1.11.1，C++20
对比系统	Redis 8.6.1
压测工具	memtier_benchmark 2.3.0

除特殊说明外，文中的对比实验均在本机回环地址 127.0.0.1 上完成，以尽量减少外部网络抖动对结果的影响。Redis 绑定到本地 6379 端口，并关闭 RDB 快照与 AOF 持久化；IdleKV 绑定到本地 4396 端口，并关闭 metrics 端口。

6.2 Redis VS IdleKV

本节基于已有的 GET/SET 压测结果，对 IdleKV 与 Redis 在基础读写负载下的表现进行对比。需要说明的是，图中横轴沿用压测脚本的比例记号，表示 SET:GET 请求比例；因此，9:1 对应写密集负载，1:1 对应读写均衡负载，1:9 对应读密集负载。

6.2.1 Get/Set

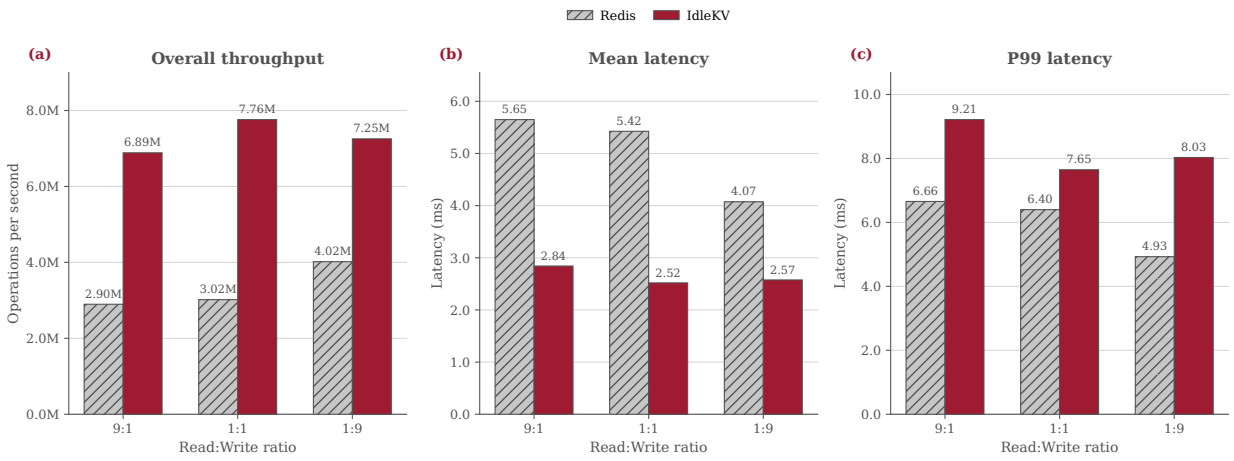

图 6.1: Get/Set 工作负载总体对比

图 6.1 展示了两套系统在三组负载下的总体吞吐、平均延迟和 P99 延迟。可以看到，IdleKV 在三组负载中都显著领先于 Redis：在 9:1、1:1 和 1:9 三组负载下，总吞吐分别达到 6.89M、7.76M 和 7.25M ops/sec，而 Redis 分别为 2.90M、3.02M 和 4.02M ops/sec，对应吞吐提升为 2.38 倍、2.57 倍和 1.81 倍。平均延迟方面，IdleKV 分别为 2.84 ms、2.52 ms 和 2.57 ms，均明显低于 Redis 的 5.65 ms、5.42 ms 和 4.07 ms，说明系统在主路径上的请求处理效率得到了稳定改善。

这一结果首先来自 Shared-Nothing 多线程架构对多核 CPU 的有效利用。Redis 的核心命令路径仍然主要由单线程串行推进，因此即使 GET 与 SET 本身足够轻量，总吞吐仍受限于单核执行能力。相比之下，IdleKV 将 key 按哈希分发到不同 shard，每个 shard 在独立线程中推进请求处理、事务注册与数据访问，从而把原本集中在单线程上的请求流分散到多个执行上下文中。对于本实验中的高并发 pipelined 负载，这种分片并行能够持续压缩队列等待时间，并把更多 CPU 周期用于实际命令执行，因此整体吞吐提升较为明显。

需要指出的是，受限于实验硬件条件，本文目前只能在 6 核 CPU 环境下对 Redis 与 IdleKV 进行对比测试。因此，图 6.1 更多反映的是 IdleKV 在当前机器规模下相对于 Redis 单线程执行模型的性能优势，而不能充分刻画系统在更多核心数量下的垂直扩展趋势。理论上，Shared-Nothing 架构应当能够随着 shard 数量和工作线程数量的增加继续释放多核并行能力，但实际扩展效果仍会受到网络栈、内存带宽、分片负载均衡、跨 shard 协调以及调度开销等因素影响。后续若能在更高核心数的服务器上补充 scaling 实验，例如逐步增加工作线程数量并观察吞吐、平均延迟和尾延迟变化，将能够更完整地评估 IdleKV 的单机扩展能力。

不过，图 6.1 也表明当前实现的尾延迟控制仍有改进空间。尽管 IdleKV 的平均延迟明显优于 Redis，但其 P99 延迟在三组负载下分别为 9.21 ms、7.65 ms 和 8.03 ms，均高于 Redis 的 6.66 ms、6.40 ms 和 4.93 ms。这说明 IdleKV 已经显著优化了大多数请求的常规执行路径，但少量请求仍可能受到 shard 队列排队、Fiber 调度、事务注册以及批量发送策略的影响，从而在尾部产生额外抖动。换言之，当前系统已经在吞吐与平均响应时间上建立明显优势，但在尾延迟稳定性方面仍需继续优化。

6.2.2 ZSet

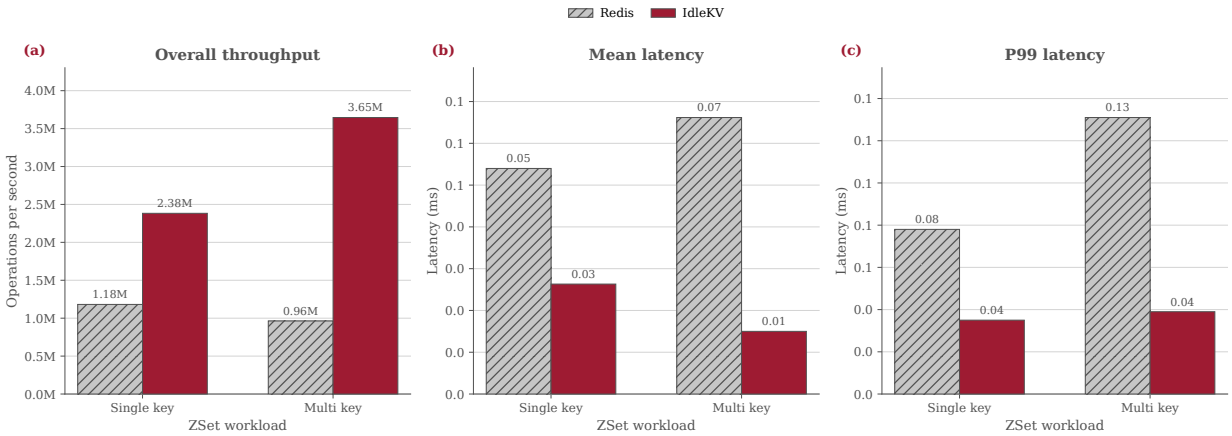

图 6.2: ZSet 工作负载总体对比

本小节展示 ZSet 的对比结果。其中两组实验底层都对应 ZSet 插入负载，只是前者持续向同一个 key 中写入 member，后者则把请求分散到多个 key 上，以观察 Shared-Nothing 分片对复合对象路径的影响。

从图 6.2 可以看到，IdleKV 在两组 ZSet 负载下吞吐提升约 2.02 倍；平均延迟分别为 0.026 ms 与 0.054 ms，IdleKV 约降低 48.7%。在 Multi key 负载下，IdleKV 的吞吐进一步达到 3.65M ops/sec，显著高于 Redis 的 0.96M ops/sec，对应吞吐提升约 3.79 倍；平均延迟则从 Redis 的 0.066 ms 降至 0.015 ms，下降约 77.3%。与此同时，P99 延迟在两组负载下也都明显优于 Redis：Single key 为 0.035 ms 对 0.078 ms，Multi key 为 0.039 ms 对 0.131 ms。

表 6.2: 单个 ZSet 键在不同数据规模下的内存占用

Key-Value 服务	数据量	内存占用
IdleKV	100,000	14.68 MB
Redis	100,000	10.35 MB
IdleKV	1,000,000	175.33 MB
Redis	1,000,000	95.64 MB

不过，从空间开销角度看，IdleKV 当前的 ZSet 实现明显重于 Redis。由表 6.2 可见，

在单个 ZSet 键包含 100,000 个 member 时，IdleKV 的内存占用为 14.68 MB，高于 Redis 的 10.35 MB；当数据规模扩大到 1,000,000 个 member 时，IdleKV 的内存占用进一步上升到 175.33 MB，而 Redis 为 95.64 MB，前者约为后者的 1.83 倍，平均每个元素要比 Redis 多用 80 bytes。

其主要原因在于 Redis 采用 SkipList + Hash Table 的方式, 对于每一个（socre, member）数据，可以只分配一份对象空间，SkipList 和 Hash Table 都只存储该对象的地址即可。但对于 IdleKV 采用的 ART 顺序索引结构,（socre，member）不会只以一整条字符串的形式存放在单个对象中，而是会随着节点分裂和路径压缩被拆散到节点前缀、边以及叶子后缀中，难以实现像 Redis 那样的优化。

6.2.3 结果讨论

从已经完成的 GET/SET 实验来看，IdleKV 当前最明确的优势体现在基础命令路径上。Shared-Nothing 分片、多线程事件循环以及连接内批量处理共同降低了单位请求的调度摊销，使系统在单机多核环境下能够稳定获得高于 Redis 的总体吞吐，并在平均延迟上保持更优表现。这说明本文在运行时层面所采用的线程模型与分片执行策略是有效的。

与此同时，实验也暴露出当前实现的若干工程短板。首先，系统虽然在平均路径上表现良好，但 Get/Set 场景下的 P99 延迟仍弱于 Redis，表明尾延迟控制、调度公平性与批量发送策略仍需继续打磨。其次基于 ART 实现的有序集合虽然在吞吐已经延迟上优于 Redis 的实现，但是内存占用要比 Redis 高近 2 倍，并且难以优化。

7 总结与展望

7.1 工作总结

本文完成了一个面向 Redis 兼容语义的主内存键值数据库原型。系统在运行时层面构建了基于 Boost.Asio 与 Boost.Fiber 的多线程事件驱动框架，在执行层面实现了 Shared-Nothing 分片架构和基于 VLL 的事务注册机制，在数据结构层面实现了基于 ART 的 ZSet 双索引结构, 并通过 Get/Set 与 ZSet 两类实验验证了整体设计的有效性。综合来看, IdleKV 已经具备了较清晰的模块边界、可运行的事务调度路径和后续继续扩展的工程基础。

7.2 不足分析与后续工作

虽然在基础命令的性能上 IdleKV 已经优于 Redis 不少，但仍存在若干明显不足与待完成工作。

- **长尾延迟严重**。实验结果表明, IdleKV 虽然在总体吞吐和平均延迟上已经优于 Redis, 但在部分基础读写负载下 P99 延迟仍然偏高，说明系统在尾部请求上的稳定性仍有不足。造成这一问题的原因可能包括 shard 任务队列排队、Fiber 调度时机、事务注册与执行阶段的额外调度，以及批量发送策略对少量请求的等待放大。后续需要进一步优化调度公平性和队列推进策略，并对关键路径中的批处理阈值、让权机制和跨 shard 协调开销进行更细粒度的分析，以在保持高吞吐的同时降低尾延迟抖动。

- **命令覆盖不足**。当前系统仅实现了 Redis 命令集中与本文实验密切相关的一部分功能, 主要覆盖字符串对象、基础系统命令以及 Sorted Set 相关命令。Hash、List、Set 等常用对象类型仍未完整实现，MULTI/EXEC 等事务型接口也尚未补齐。因此，IdleKV 目前更接近一个用于验证核心架构和关键数据结构的原型系统，后续仍需要继续扩展命令集合、完善对象语义，并提升与 Redis 生态工具的兼容程度。

- **持久化缺失**。当前系统尚未实现数据持久化能力，重启后无法恢复内存数据。Redis 通常结合 AOF 与 Fork Snapshot 实现持久化，但 Fork Snapshot 在大内存实例中可能因写时复制带来短时间内存占用激增，并进一步影响尾延迟稳定性。后续可以参考 DragonflyDB 的 Forkless Snapshot 设计，在避免大规模 fork 开销的前提下实现一致性快照，并进一步结合增量日志机制提升系统恢复能力[22]。

- **集群能力缺失**。一个完整的生产级内存数据库不仅需要提升单机多核利用率，还需要支持分布式集群部署，以满足水平扩展、故障恢复和高可用需求。当前 IdleKV 仍只关注单机内的分片执行与事务协调，尚未实现跨节点分片、数据迁移、副本同步、故障检测和自动主从切换等机制。后续若要面向更大规模负载，需要在保持单机执行效率的基础上进一步设计集群协议和复制机制。

8 致谢

参考文献

[1] TESSIER J. In-memory databases: The foundation of real-time ai and analytics[EB/OL]. (2025-10-23)[2026-04-20]. https://redis.io/blog/in-memory-databases-the-foundation-of-real-time-ai-and-analytics/.

[2] OUSTERHOUT J, AGRAWAL P, ERICKSON D, et al. The case for ramclouds: Scalable high-performance storage entirely in dram[J]. SIGOPS Operating Systems Review, 2009, 43(4): 92-105.

[3] FAN B, ANDERSEN D G, KAMINSKY M. Memc3: Compact and concurrent memcache with dumber caching and smarter hashing[C/OL]//10th USENIX Symposium on Networked Systems Design and Implementation (NSDI 13). Lombard, IL: USENIX Association, 2013: 371-384. https://www.usenix.org/conference/nsdi13/technical-sessions/presentation/fan.

[4] LIM H, HAN D, ANDERSEN D G, et al. Mica: A holistic approach to fast in-memory key-value storage[C/OL]//11th USENIX Symposium on Networked Systems Design and Implementation (NSDI 14). Seattle, WA: USENIX Association, 2014: 429-444. https://www.usenix.org/conference/nsdi14/technical-sessions/presentation/lim.

[5] CHANDRAMOULI B, PRASAAD G, KOSSMANN D, et al. Faster: A concurrent key-value store with in-place updates[C/OL]//Proceedings of the 2018 International Conference on Management of Data. Houston, TX, USA: Association for Computing Machinery, 2018: 275-290. https://www.microsoft.com/en-us/research/publication/faster-concurrent-key-value-store-place-updates/.

[6] BERNSTEIN P A, GOODMAN N. Concurrency control in distributed database systems [J]. ACM Comput. Surv., 1981, 13(2): 185–221.

[7] GRAY J N, LORIE R A, PUTZOLU G R. Granularity of locks in a shared data base[C]// VLDB ’75: Proceedings of the 1st International Conference on Very Large Data Bases. New York, NY, USA: Association for Computing Machinery, 1975: 428–451.

[8] REED D P. Naming and synchronization in a decentralized computer system[R]. USA: Massachusetts Institute of Technology, 1978.

[9] KUNG H T, ROBINSON J T. On optimistic methods for concurrency control[J]. ACM Trans. Database Syst., 1981, 6(2): 213–226.

[10] REN K, THOMSON A, ABADI D J. Vll: a lock manager redesign for main memory database systems[J]. The VLDB Journal, 2015, 24(5): 681–705.

[11] RAO J, ROSS K A. Making b+-trees cache conscious in main memory[C]//Proceedings

of the 2000 ACM SIGMOD International Conference on Management of Data. New York, NY, USA: Association for Computing Machinery, 2000: 475-486.

[12] PUGH W. Skip lists: a probabilistic alternative to balanced trees[J]. Communications of the ACM, 1990, 33(6): 668-676.

[13] MAO Y, KOHLER E, MORRIS R. Cache craftiness for fast multicore key-value storage [C/OL]//Proceedings of the 7th ACM European Conference on Computer Systems. Bern, Switzerland: Association for Computing Machinery, 2012: 183-196. https://read.seas.harvard.edu/~kohler/pubs/mao12cache.pdf.

[14] LEIS V, KEMPER A, NEUMANN T. The adaptive radix tree: Artful indexing for main-memory databases[C]//ICDE '13: Proceedings of the 2013 IEEE International Conference on Data Engineering (ICDE 2013). USA: IEEE Computer Society, 2013: 38–49.

[15] HERLIHY M P, WING J M. Linearizability: a correctness condition for concurrent objects [J]. ACM Trans. Program. Lang. Syst., 1990, 12(3): 463–492.

[16] Jepsen-io. Elle[EB/OL]. (2020-02-23)[2026-04-20]. https://github.com/jepsen-io/elle.

[17] STONEBRAKER M. The case for shared nothing[C/OL]//High Performance Transaction Systems Workshop. 1985. https://api.semanticscholar.org/CorpusID:260427782.

[18] Boostorg. Boost.asio[EB/OL]. 2026[2026-04-21]. https://github.com/boostorg/asio.

[19] Boostorg. Boost.fiber[EB/OL]. 2026[2026-04-21]. https://github.com/boostorg/fiber.

[20] COLLET Y. xxhash - extremely fast non-cryptographic hash algorithm[EB/OL]. 2012. https://xxhash.com/.

[21] LEIJEN D, ZORN B G, DE MOURA L M. Mimalloc: Free list sharding in action[C/OL]// Asian Symposium on Programming Languages and Systems. 2019. https://api.semanticscholar.org/CorpusID:198363081.

[22] DragonflyDB. Rdb save design[EB/OL]. 2026[2026-04-24]. https://github.com/dragonflydb/dragonfly/blob/main/docs/rdbsave.md.